164099

PUFFERFISH NEST BUILDER GAME

1. Introduction

The project is a 2D HTML5 Canvas-based game inspired by the real nesting behaviour of male pufferfish discovered near Amami-Oshima Island in Japan. The application demonstrates graphics pipeline concepts, including the application stage, geometry stage, and rasterization stage, using JavaScript and the Canvas API.

2. Inspiration for the Game

Divers off the coast of Amami-Oshima Island in Japan noticed mysterious, perfect geometric circles etched into the sandy seafloor. In 2013, a Japanese photographer finally caught a tiny pufferfish about 12 cm long, forming what looked like a crop circle underwater. It spent about 7-9 days working and swimming in patterns, collecting shells and sand fragments. This was being done by a male pufferfish to impress a female.

The ridges and valleys of the circular nests would manipulate the water current to create a calm zone, safe and still, where eggs could be laid and fertilized without being swept away. The shells along the peaks act as the visual signal of the male's quality and dedication. If a female swims by and is impressed, she enters the center, they mate, she lays her eggs there, and leaves. The male then guards the nest alone until the eggs hatch. The male then leaves the nest and builds a new one.

3. The Goal of the Game

The player is the male pufferfish. The mission is to complete the nest by collecting shells and sand fragments and placing them in the nest pattern before the shark catches the player. Complete the nest to attract the female and win the round.

4. Components of the Game

The pufferfish

- Controlled using the arrow keys
- Can collect one material at a time
- Grows as nest progress increases

The nest

- The nest sits at the center of the seafloor.
- It is a geometric circular pattern.
- It starts as an empty, faint outline on the sand, and as you deposit more materials, the nest fills in.

- The nest, when done, looks like a beautiful glowing geometric circle on the seafloor.

Safe Zone

- The nest emits a glow radius, which is the safe zone.
- Inside this radius, the shark will not kill the fish.
- The bigger and more complete the nest is, the wider the safe radius is.

The Collectibles

- The pufferfish can collect sand fragments, small shells, or rare coloured shells.
- The sand fragments are close to the safe zone, hence low nest progress.
- The small shells are at a medium distance from the safe zone, hence medium progress.
- The rare colored shells are further at risk and are greater progress.

The Shark

- The shark patrols the water and moves around. If the shark senses the fish, it moves near the fish and tries to catch it. The fish must go back to the safe area zone to be safe, and the shark will turn and go away.
- Getting caught by the shark makes the fish lose one life. It has three lives in total. Losing a life also drops the material you were carrying.

Behaviour

- Collectibles are scattered randomly across the seafloor at the start.
- It is collected by simply swimming over them
- You carry one at a time. You must return to the nest to deposit before collecting again.
- When carrying a material, the pufferfish visually holds it underneath its body.

Win and Lose Conditions

- To win, the nest is fully complete, and a female fish swims in.
- To lose, the three lives are lost, and the game restarts.

IMPLEMENTATION

Technology Used

1. HTML5 Canvas for rendering graphics
2. CSS for styling
3. JavaScript for game logic and animation

Graphics Pipeline

The Application Phase

- It defines and controls what should happen on the screen.
- It involves writing code, creating objects, handling user input, and controlling behavior and interaction.

The application stage handles all gameplay logic and object behaviour before rendering occurs. In this project, the application stage includes player movement, shark behaviour, collision detection, collectible spawning, score updates, and win/loss conditions. Functions such as updatePlayer(), updateShark(), and updateCollectibles() update the game state every frame before objects are rendered to the canvas.

The Geometry Phase

- The geometry phase is responsible for defining and positioning objects mathematically before they are rendered onto the screen.
- It involves creating shapes using coordinates, curves, circles, and transformations such as translation, rotation, and scaling.

The geometry stage defines all visual objects in the game, including the pufferfish, shark, nest, collectibles, rocks, and seaweed. Shapes are created using Canvas geometry functions such as ctx.arc(), ctx.ellipse(), ctx.bezierCurveTo(), and ctx.quadraticCurveTo(), which mathematically define the structure of each object before rendering.

Objects are positioned using coordinate transformations such as translate(), rotate(), and scale(), allowing them to move and animate within the game world. For example, the shark and pufferfish are translated to their current positions each frame, while rotation is used to animate movement like the shark's tail sway.

The nest is generated using circular arc calculations to form concentric rings, while trigonometric functions (sin and cos) are used to place shells and spokes evenly around the structure.

The RasterizationPhase

- The rasterization phase converts geometric shapes (primitives) into pixels on the screen.
- It determines which pixels are covered by each shape and assigns final colors to them.

The rasterization stage is responsible for turning the geometric objects (such as the pufferfish, shark, nest, and collectibles) into visible pixels on the canvas. After the geometry stage defines shapes using functions like ctx.arc() and ctx.ellipse(), the rasterization stage fills and strokes these shapes using Canvas rendering methods such as fill(), stroke(), fillRect(), and fillText().

This stage also handles visual effects like gradients, glow, and shading, which determine the final appearance of objects on the screen. Each frame is redrawn, meaning pixel data is continuously updated in real time to create smooth animation. This phase produces the final image seen by the player, combining all shapes, colors, and effects into the completed underwater scene.